

Refactorización del backend Calendario

Optimización de rendimiento, corrección de errores, limpieza estructural y documentación bilingüe

Proyecto: CCASolutions / Calendario (Spring Boot 4.0.2, Java 17) **Rama:** backup_claudeveloper_2026-09-06 **Base:** 6e1088f
Fecha: 6 de septiembre de 2026

9,8× más rápida la conversión de una fecha actual (491 ms → 50 ms)	16 errores corregidos, uno de ellos una caída con HTTP 500	175 fechas comparadas byte a byte: respuesta idéntica a la original	~10⁹ comparaciones eliminadas en cada poblado de la BD	120 clases documentadas en inglés y español
--	--	---	---	---

1. Resumen

Se ha revisado el proyecto completo (29 clases de servicio, controladores, repositorios y entidades) con tres objetivos: hacerlo más rápido, corregir los defectos encontrados por el camino y dejarlo documentado de forma que se pueda leer sin conocer previamente el dominio. El trabajo se ha hecho de forma conservadora: **salvo en los puntos que se documentan como corrección de un error, la respuesta del API es exactamente la misma que antes**, y eso se ha comprobado de forma automática comparando la salida de las dos versiones sobre 175 fechas repartidas entre los años 1401 y 2098.

El resultado principal es que el endpoint de conversión de fechas ha pasado de tardar entre 25 y 492 ms (según lo lejos que estuviera la fecha del año de referencia del calendario) a tardar entre 20 y 52 ms de forma prácticamente plana. Para las fechas que realmente se consultan a diario, la mejora es de casi diez veces.

2. Cómo se ha verificado

Antes de tocar nada se levantó la versión original en el puerto 8082 y la refactorizada en el 8081, ambas contra la misma base de datos MySQL de desarrollo, y se compararon las respuestas JSON completas:

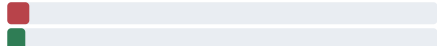
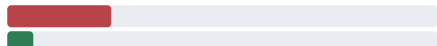
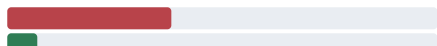
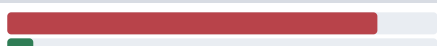
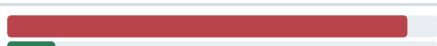
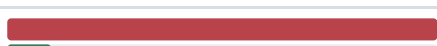
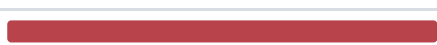
COMPROBACIÓN	ALCANCE	RESULTADO
Comparación byte a byte de la respuesta JSON	175 fechas (5 por año, en 35 años entre 1401 y 2098)	175 idénticas / 0 distintas
Fechas límite del rango soportado	2099-12-31, primer y último año con datos	Corregida (antes HTTP 500)
Tests unitarios nuevos de las utilidades de búsqueda	9 tests, contrastados contra la lógica original por fuerza bruta	9 / 9 correctos
Suite completa (<code>mvn clean test</code>)	Incluye la carga del contexto de Spring	10 / 10 correctos
Prueba de humo del endpoint de poblado	Con todas las tablas llenas: se ejercitan las guardas sin escribir	HTTP 200, sin escrituras
Repetición de la comparación tras documentar	Las mismas 175 fechas, ya con todo el Javadoc añadido	175 idénticas / 0 distintas

Lo que no se ha podido ejecutar

La ruta de poblado de la base de datos no se ha ejecutado de principio a fin, porque requiere partir de una base de datos vacía y hacer varios miles de llamadas a la API externa de OPALE. Sus cambios están verificados por revisión, por los tests de las utilidades de búsqueda en las que se apoyan y por la prueba de humo de las guardas, pero no por una ejecución real. Es lo primero que conviene probar en un entorno desechable.

3. Rendimiento del endpoint de conversión

Medición con `curl` reutilizando conexión, media de 30 peticiones tras 10 de calentamiento, sobre la misma base de datos y la misma máquina. La columna «antes» es el código de `6e1088f` sin tocar.

FECHA CONSULTADA	ANTES	DESPUÉS	MEJORA	COMPARATIVA
1440-06-01	25 ms	20 ms	1,3×	
1600-06-01	119 ms	31 ms	3,8×	
1700-06-01	186 ms	35 ms	5,3×	
1800-06-01	440 ms	50 ms	8,8×	
1900-06-01	425 ms	28 ms	15,2×	
2000-06-01	456 ms	52 ms	8,8×	
2026-09-06 (hoy)	491 ms	50 ms	9,8×	
2099-06-01	492 ms	46 ms	10,7×	

De dónde venía el coste

El tiempo crecía de forma lineal con la distancia entre la fecha consultada y el metono inercial apofasal remoto de referencia (año 1433). El motivo estaba en `UtilsServiceImpl.getDatosCosmicos`:

```
datos.setLunas(lunasRepository.findByDateBetween(  
    lastMetonIApofasalRemoto.getDate().minusYears(1), // año 1432  
    date0.plusYears(1))); // año 2027
```

Es decir, cada petición cargaba como entidades JPA **todas las fases lunares de casi seiscientos años**: más de 72.000 filas de las 103.802 que tiene la tabla, para después recorrerlas cinco veces en memoria. De todo ese intervalo el cálculo sólo necesitaba dos cosas: las fases de los años inmediatamente anteriores y posteriores a la fecha, y las lunas nuevas selectas (los aponovos) del intervalo completo, que son unas 450 filas.

La consulta se ha partido en esas dos piezas y el recuento de lunas nuevas desde el último aponovo se resuelve ahora con un `COUNT` sobre el índice `(nueva, date)` en vez de recorriendo la lista. El conjunto de filas que ven los cálculos es exactamente el mismo, y de ahí que las 175 respuestas comparadas sean idénticas.

4. Errores encontrados y corregidos

01 Caída con HTTP 500 en las fechas sin solsticio posterior

SeasonsServiceImpl.getVAUSeason · FestividadesServiceImpl · NotableEventServiceImpl

Las variables `lastSoe` y `nextSoe` se inicializaban con `new SolsticiosYEquinocciosEntity()` en lugar de `null`. El control `if (lastSoe != null && nextSoe != null)` nunca fallaba, así que cuando no había solsticio posterior se llamaba a `ChronoUnit.SECONDS.between()` con una fecha nula.

Comprobado: GET /api/conversiontovau/selected?date=2099-12-31 devolvía HTTP 500 – `NullPointerException: temporal`. Tras el arreglo devuelve 200 con la conversión completa. Afecta a todo el final del rango soportado.

02 Se comprueba el apoperi anterior y se usa el próximo

NotableEventServiceImpl.getEventoProximo

El control era `if (...getApoperiAnterior() != null)` pero dentro se leía `...getApoperiProximo().getDate()`. Con un apoperi anterior pero ninguno posterior, `NullPointerException`.

Impacto: caída del endpoint en el extremo superior del rango de datos.

03 Los eclipses solares se guardaban marcados como lunares

EclipsesServiceImpl.actualizarEclipsesSolaresDelAnyo

Al construir la fila de la tabla histórica `all_eclipses` se hacía `allEclipseParaDB.setDeLuna(true)` en el método de eclipses solares (copia y pega del método lunar). Corregido a `setDeSol(true)`.

Impacto: la tabla `all_eclipses` queda con `de_sol = 0` en todas sus filas. Los datos ya cargados con la versión anterior están mal y habría que regenerarlos.

04 Buscadores que devuelven una entidad vacía en vez de null

MetonsServiceImpl.getLastMetonIApofasalRemoto / getLastMetonINForDate ·
LunasServiceImpl.getComportamientoLuna · MonthServiceImpl

Los buscadores partían de `new MetonsEntity()` y devolvían ese objeto vacío cuando no encontraban nada.

`UtilsServiceImpl` comprobaba `!= null`, el control nunca saltaba y el fallo aparecía varias llamadas después, al leer `.getDate()` del objeto vacío. Ahora devuelven `null` y quien los llama lo trata.

Impacto: el mismo patrón se repetía en cuatro clases; además hacía inútil todo el bloque de mensajes de error de `getDatosCosmicos`.

05 || donde debía ir &&

MonthServiceImpl.getVAUMonth

`if (lastLNBeforeNextSOE != null || firstLNAfterLastSOE != null)` y dentro se desreferencian las dos variables. Basta con que una sea nula para provocar un `NullPointerException`.

06 El apellido del métono apórico se decidía con el contador equivocado

MetonsServiceImpl.getVAMeton

El bloque que asigna «(Selecto)» o «(Invertido)» al métono invernal apórico comprobaba `yearOfTheMetonIN != 0` en vez de `yearOfTheMetonIA != 0`.

Impacto: apellido incorrecto en el métono apórico cuando ambos contadores no coinciden.

07 `setNuevo(true)` incondicional pisaba el valor real

`CasalerosServiceImpl.getCasalero`

Se hacía `casaleroDTO.setNuevo(metono.isNuevo())` y cinco líneas más abajo `casaleroDTO.setNuevo(true)`, de modo que todos los casaleros metónicos salían como «nuevo».

08 Accesos a `.get(0)` sin comprobar la lista

`EclipenosServiceImpl.getVAUEclipeno` · `MetonsServiceImpl.getVAUMeton` y `getMetonoInvernalApofasalRemoto`

Tres listas se construían filtrando y se leía su primer elemento sin comprobar que no estuvieran vacías:

`IndexOutOfBoundsException` para cualquier fecha sin fenómenos entre el eclípeno de referencia y la fecha.

09 Una festividad sin calcular contaba como «festividad de hoy»

`FestividadesServiceImpl.getFestividades`

La clasificación era `if (festividad.getDiasDeDiferenciaConDate() == 0) → actual`, y el DTO nace con ese contador a cero y la fecha a `null`. Cualquier festividad que no llegara a calcularse entraba en el grupo de las de hoy. Ahora se descartan las que no tienen fecha.

10 Controles de URL de API que nunca saltaban

`EclipsesServiceImpl.poblareEclipsesFromOpale`

`apiEclipsesLunares` y `apiEclipsesSolares` se inicializaban a `""` y luego se comprobaba `!= null`. Si faltaba la fila en `datos`, se llamaba a la API con una URL vacía en lugar de avisar. Ahora se inicializan a `null` y el control funciona.

11 `NullPointerException` al arrancar el poblado sin la fila `APG`

`ApogeosYPerigeosLunaServiceImpl.poblareApogeosFromOpale`

Era el único de los cinco poblados que llamaba a `apiGetApogeosUrl.getValor()` sin comprobar antes que la fila existiera.

12 `findAll()` sin `ORDER BY` usado por posición

`MidsisonServiceImpl.poblareMidsison`

Los midsisons se calculan emparejando cada solsticio o equinoccio con el siguiente: `allSoesFromDB.get(i)` con `get(i+1)`. La lista venía de un `findAll()` sin orden garantizado. Sustituido por `findAllOrderByDateAsc()`.

Impacto: latente. Hoy MySQL devuelve las filas en orden de clave primaria y coincide con el cronológico, pero no está garantizado y un cambio de plan de ejecución produciría midsisons mal calculados en silencio.

13 Los endpoints de descarga descartaban el resultado

`DownloadController.getPDF` y `getCalendar`

Se llamaba a `this.downloadService.getPDF()` sin recoger el valor devuelto y se respondía siempre `new byte[0]` con HTTP 200. Además, `getcalendar` recibía el DTO como `@RequestParam`, y Spring no puede construir un objeto complejo a partir de un parámetro de query. Ahora se devuelve el contenido real, el DTO llega como `@RequestBody` y, mientras el servicio siga sin implementar, se responde `501 Not Implemented` en vez de un 200 vacío.

14 Semana y día sin luna nueva anterior

`LunasServiceImpl.getVauWeekAndDay` · `DaysServiceImpl.getDiasASumarALaLunaNueva`

Si no había ninguna luna nueva anterior a la fecha, el contador de días se quedaba en `Long.MAX_VALUE` y se consultaba el día número `Long.MAX_VALUE - 21`, que devuelve `null` y revienta al leer su nombre. Lo mismo en `getDiasASumarALaLunaNueva`, que no comprobaba ni la semana ni el día antes de usarlos.

15 «hace 9223372036854775807 días»

`NotableEventServiceImpl.getEventoPasado` / `getEventoProximo`

Cuando no había ningún fenómeno a un lado, el mínimo se quedaba en `Long.MAX_VALUE` y se componía el texto con ese número. Ahora se devuelve cadena vacía.

16 Mensajes de error invertidos o equivocados

`EclipenosServiceImpl.poblareEclipenos` · `EclipsesServiceImpl.actualizarEclipsesLunaresDelAnyo`

El primero avisaba «No hay métonos en la base de datos» precisamente en la rama en la que sí los había (`else if (!metonos.isEmpty())`). El segundo registraba «Actualizando los eclipses solares» dentro del método de eclipses lunares, dos veces.

5. Optimización del poblado de la base de datos

El proceso de poblado cruzaba tablas completas con bucles anidados. Con los recuentos reales de la base de datos actual (103.802 lunas, 29.196 apogeos y perigeos, 9.882 eclipses, 8.396 solsticios y equinoccios, 8.395 midsisons, 1.755 métonos, 221 eclípenos) esos cruces suponen del orden de **cinco mil millones de comparaciones**.

Como todos los cruces buscan lo mismo —fenómenos que caen dentro del mismo día sideral (86.164 s)— se han sustituido por un índice ordenado por fecha con búsqueda binaria (`IndiceTemporal`), que sólo recorre las pocas filas realmente cercanas.

PROCESO	COMPARACIONES ANTES	DESPUÉS
<code>updateLunasYApoperisConSelecto0Invertido</code> apoperis × lunas	~3.030.000.000	29.196 búsquedas binarias
<code>poblateMidsison</code> midsisons × (lunas + apoperis + eclipses)	~1.200.000.000	3 × 8.395 búsquedas binarias
<code>poblateMetonos</code> soes × (lunas + apoperis)	~1.118.000.000	2 × 8.396 búsquedas binarias
<code>poblateMetonos</code> (marcado de apofasales)	~51.000.000	1.755 búsquedas en ventana

Consultas y escrituras

PROCESO	ANTES	DESPUÉS
<code>poblateEclípenos</code>	1.755 <code>SELECT ... findByYear</code> , uno por métono, y un <code>INSERT</code> por eclípeno	1 <code>SELECT</code> + agrupación en memoria + <code>saveAll</code>
<code>poblateCasaleros</code>	~660 <code>SELECT</code> (2 ó 3 por eclípeno) + 221 <code>INSERT</code> sueltos	2 <code>SELECT</code> + búsqueda binaria + <code>saveAll</code>
<code>updateLunasYApoperis...</code>	Hasta 2 <code>UPDATE</code> por coincidencia, dentro del bucle	Dos <code>saveAll</code> al final
<code>poblateLunasFromOpale</code>	Un <code>INSERT</code> por fase lunar (más de 200.000)	<code>saveAll</code> por año
<code>poblateEclipses...</code> y <code>poblateSolsticios...</code>	Un <code>INSERT</code> por fila	<code>saveAll</code> por año
Comprobación «¿ya hay datos?»	<code>findAll()</code> de la tabla entera para mirar si está vacía	<code>count()</code>

Además se ha activado el envío por lotes de Hibernate (`hibernate.jdbc.batch_size=500`, `order_inserts`, `order_updates`) y `rewriteBatchedStatements=true` en la URL de MySQL, sin lo cual `saveAll` seguiría enviando una sentencia por fila.

6. Cambios estructurales

Tres utilidades que eliminan la duplicación

- `Utils/Vecindad` — «el fenómeno de hoy, el anterior más próximo y el siguiente más próximo». Sustituye a los bucles `diasMinimosDeDiferenciaEntreXPasadoYDate` que estaban repetidos catorce veces. Recorre la lista una

sola vez y convierte cada fecha una sola vez, en lugar de las tres o cuatro conversiones por elemento del original.

- `Utils/IndiceTemporal` — índice ordenado con búsqueda binaria para «qué hay a menos de un día sideral de este instante» y «cuál es el primero posterior a esta fecha».
- `Utils/FechasApi` — troceo de las fechas ISO de OPALE, que llegan con signo para los años anteriores al 1 (`-4700-01-01T...`) y no se pueden parsear directamente. Estaba copiado literalmente en cuatro sitios.

Caché de las tablas de referencia

Cada conversión de fecha lanzaba entre cinco y ocho consultas a tablas de menos de veinte filas (días, semanas, meses, estaciones y festividades) que sólo cambian al repoblar. Se ha añadido `TablasReferenciaService` con `@Cacheable` sobre un `ConcurrentMapCacheManager`, y `DBServiceImpl` vacía la caché al terminar el poblado.

Simplificación de `NotableEventServiceImpl`

Era la clase más grande del proyecto (1.015 líneas). Contenía seis bucles idénticos salvo por el tipo del elemento y **tres copias literales del mismo bloque de noventa líneas** que construye el midsison, una por cada caso (pasado, actual y futuro). La lógica ha quedado en 651 líneas con el mismo comportamiento (844 ya con la documentación).

En `MonthServiceImpl` se ha eliminado además un bucle completo sobre las lunas llenas cuyos resultados no se usaban en ninguna parte (variables marcadas con «ya tendrá utilidad»), y dos ramas `if (caeEnLunaNueva)` inalcanzables porque el caso ya se había resuelto y devuelto antes.

Registro de trazas

Había **113 llamadas a `System.out.println`**, algunas dentro de bucles que se ejecutan cientos de miles de veces durante el poblado. `System.out` está sincronizado y no se puede filtrar por nivel. Todas se han sustituido por SLF4J con el nivel adecuado (`debug` para las trazas de bucle, `info` para el progreso, `warn` y `error` para los problemas), y las excepciones se registran como excepción, con su traza, en lugar de concatenarlas a una cadena.

Persistencia y configuración

- `@Transactional(readOnly = true)` en la ruta de lectura: evita el trabajo de *dirty checking* de Hibernate sobre las miles de entidades cargadas por petición.
- `spring.jpa.open-in-view=false`: la conexión se devuelve al pool al salir del servicio, no al serializar la respuesta.
- Índice `idx_midsison_date` declarado en la entidad. Era la única tabla consultada por fecha sin índice; se ha verificado que `ddl-auto=update` lo ha creado.
- Usuario, contraseña, host y puerto de la base de datos leídos de variables de entorno con los valores actuales como valor por defecto, para no dejar la contraseña escrita en el repositorio (`${DB_PASSWORD:1234}`).
- Los controladores se han reordenado con salidas tempranas en lugar de un `if` anidado a cuatro niveles con una variable de estado mutable. **Los códigos HTTP no se han cambiado** salvo en los dos endpoints de descarga, para no romper el frontal.

7. Documentación bilingüe

Todo el código lleva ahora Javadoc en inglés y en español, en ese orden dentro del mismo bloque, de modo que se pueda leer en cualquiera de los dos idiomas sin duplicar ficheros ni mantener dos versiones separadas. El formato es siempre el mismo:

```
/**
 * EN: Most recent winter apofasal remote meton on or before the date: winter solstice with
 * a new moon at apogee, all within one sidereal day. It is the reference the VAU meton
 * counters hang from.
 * ES: Métono invernal apofasal remoto más reciente en la fecha o anterior: solsticio de
 * invierno con luna nueva en apogeo, todo dentro de un día sideral. Es la referencia de la
 * que cuelgan los contadores de métono VAU.
 *
 * @param allMetons EN: metons to search. / ES: métonos donde buscar.
 * @param date      EN: upper bound, inclusive. / ES: cota superior, incluida.
 * @return EN: the meton, or {@code null} if there is none. / ES: el métono, o {@code null}
 *         si no hay ninguno.
 */
```

QUÉ SE HA DOCUMENTADO	ELEMENTOS	CONTENIDO
Métodos de servicios, controladores, repositorios, configuración y utilidades	~240	Qué hace, por qué se hace así cuando no es evidente, y cada parámetro y valor devuelto
Interfaces de servicio	20	Qué concepto del calendario representa cada una
Repositorios	17	Traducción de los nombres derivados de Spring Data, que son largos y crípticos
DTOs	37	Qué representa cada uno y qué significan sus campos de dominio
Entidades JPA	17	Qué tabla es, qué guarda y cómo se relaciona con las demás
Total de bloques Javadoc	410	En 120 de 120 ficheros <code>.java</code>

Qué aporta el Javadoc frente a los nombres

El proyecto usa un vocabulario propio —métono, eclípeno, apoperi, midsison, aponovo, apofasal, selecto, invertido, remoto— que no significa nada fuera de él. La documentación explica cada término allí donde aparece, de modo que ya no hace falta reconstruirlo leyendo el código. Por ejemplo, la clase `EclípenosEntity` ahora empieza así:

```
/**
 * EN: One eclípeno, table `eclípenos`: a fasal meton that also coincides with an
 * eclipse. It is the rarest cycle of the calendar, around two hundred rows in two
 * thousand years, and the origin every other VAU counter hangs from.
 * ES: Un eclípeno, tabla `eclípenos`: un métono fasal que además coincide con un
 * eclipse. Es el ciclo más excepcional del calendario, unas doscientas filas en dos
 * mil años, y el origen del que cuelgan todos los demás contadores VAU.
 */
```

Criterio seguido con los accesores

Los DTOs y las entidades tienen entre los dos **707 getters y setters** triviales (`getId()` , `setDate(...)`).

Documentarlos uno a uno en dos idiomas habría añadido unas 2.800 líneas sin aportar nada y habría enterrado las clases en ruido, que es justo lo contrario de lo que se busca. En su lugar cada DTO y cada entidad lleva un Javadoc de clase que explica qué representa y qué significan sus campos de dominio, con lo que los accesorios se leen solos. Si prefieres que también lleven Javadoc uno a uno, es un cambio mecánico que se puede hacer en cualquier momento.

La documentación no ha cambiado el comportamiento

Tras añadir los 410 bloques se repitió la comparación completa contra la versión original: **175 fechas, 175 respuestas idénticas**, y la suite de tests sigue en 10 de 10.

8. Puntos que conviene revisar

Prioridad de festividades cuando coinciden varias el mismo día

`FestividadesServiceImpl.getFestividadActual` recorre un array llamado `prioridad` sin cortar al encontrar la primera coincidencia, de modo que *gana la última*, no la primera. Puede ser intencionado (el último elemento es el cambio de eclípeno invernal apofasal remoto, el fenómeno más excepcional) o puede ser un bucle al que le falta el `break`. Al no poder deducirlo con certeza

se ha conservado el comportamiento exacto

, escribiéndolo ahora de forma explícita (recorrido inverso con salida en la primera coincidencia) y documentando el criterio. Conviene confirmar cuál es el orden querido.

El midsison se calcula dos veces

La tabla `midsison` tiene los 8.395 midsisons ya calculados y persistidos, y `getDatosCosmicos` los carga en cada petición, pero `NotableEventServiceImpl` y `FestividadesServiceImpl` los vuelven a calcular en memoria a partir de los solsticios. La lista cargada de la base de datos sólo se usa para comprobar que no está vacía. Unificarlo simplificaría bastante las dos clases, pero las reglas de `selecto` e `invertido` difieren ligeramente entre las dos versiones, así que no se ha tocado sin poder contrastar cuál es la correcta.

Seguridad

Los tres controladores llevan `@CrossOrigin("*")`, y el endpoint de poblado se protege con una única contraseña, cuyo valor por defecto (`adminrest`) está escrito en el código. No es un fallo introducido ni corregido aquí, pero conviene tenerlo presente antes de exponer el servicio.

Fechas fuera de rango

Una fecha fuera del rango soportado sigue devolviendo HTTP 200 con `fechaEncontrada: false`. Devolver un 400 sería más correcto, pero cambiar el código de estado puede romper el frontal, así que se ha dejado como estaba.

9. Ficheros tocados

TIPO	FICHEROS
Servicios reescritos o revisados	Los 19 <code>ServiceImpl</code>
Controladores	<code>DatesController</code> , <code>DBController</code> , <code>DownloadController</code>
Repositorios	<code>LunasRepository</code> (2 consultas nuevas), <code>SolsticiosYEquinocciosRepository</code> (1), <code>DaysRepository</code> y <code>WeeksRepository</code> (tipos de parámetro corregidos); los 17 documentados
Entidades	<code>MidsisonEntity</code> (índice por fecha); las 17 documentadas
DTOs	Los 37, documentados
Configuración	<code>ClasesBean</code> , <code>application.properties</code>
Clases nuevas	<code>Utils/Vecindad</code> , <code>Utils/IndiceTemporal</code> , <code>Utils/FechasApi</code> , <code>TablasReferenciaService</code> y su implementación
Tests nuevos	<code>IndiceTemporalTest</code> (5), <code>VecindadTest</code> (4)

Estado

Compila, arranca contra la base de datos de desarrollo, pasa los 10 tests y devuelve exactamente la misma respuesta que la versión anterior en las 175 fechas comparadas, más una fecha adicional en la que la versión anterior fallaba. Nada se ha subido ni se ha hecho commit: los cambios están en el árbol de trabajo de la rama `backup_claudeveloper_2026-09-06`.

Informe generado el 6 de septiembre de 2026 · Mediciones tomadas con la base de datos `calendar_db` (MySQL 9.4) en la máquina de desarrollo · Base de comparación: commit `6e1088f`